The Python programming language

Ezequiel López-Rubio

Department of Computer Languages and Computer Science

University of Málaga, Spain

Contents

- Introduction
- Fundamentals
- Flow control
- Data structures
- Modularity
- Conclusion

1. INTRODUCTION

1. Introduction

Overview

- Python is a general purpose interpreted, multi-platform programming language:
  - Easy to learn, yet powerful
  - Efficient high-level data structures
  - Simple but effective object-oriented model
  - Elegant syntax
  - Dynamic typing
- These features make it one of the most used programming languages in the world

1. Introduction

Python flavors

- Version 2 of the language is outdated
  - Here we focus on version 3
- Python can be used in several ways:
  - From the command line of the Python interpreter
  - Within an Integrated Development Environment such as Sypder or PyCharm (.py files)
  - As Python source code cells in an interactive Jupyter notebook (.ipynb files, Google Colab)

2. FUNDAMENTALS

2. Fundamentals

Data model

- All data in a Python program is represented by objects
  - Even integers are objects!
- Every object has an identity, a type and a value
  - An object's identity never changes once it has been created; it is the object's address in memory
  - The 'is' operator compares the identity of two objects
  - The id() function returns an integer representing its identity
  - The type() function returns the type of the object

2. Fundamentals

Data model

- There are many builtin numeric types. They are immutable; once created their value never changes:
  - Integers (int). These represent numbers in an unlimited range
  - Booleans (bool). These represent the truth values False and True
  - Floating-point numbers (float). These represent machine-level double precision floating point numbers
- Objects of immutable sequence types cannot be changed after they have been created:
  - Strings (str). A string is a sequence of values that represent Unicode code points
  - Tuples (tuple). The items of a tuple are arbitrary Python objects
- Objects of mutable sequence types can be changed after they have been created:
  - Lists (list). The items of a list are arbitrary Python objects

2. Fundamentals

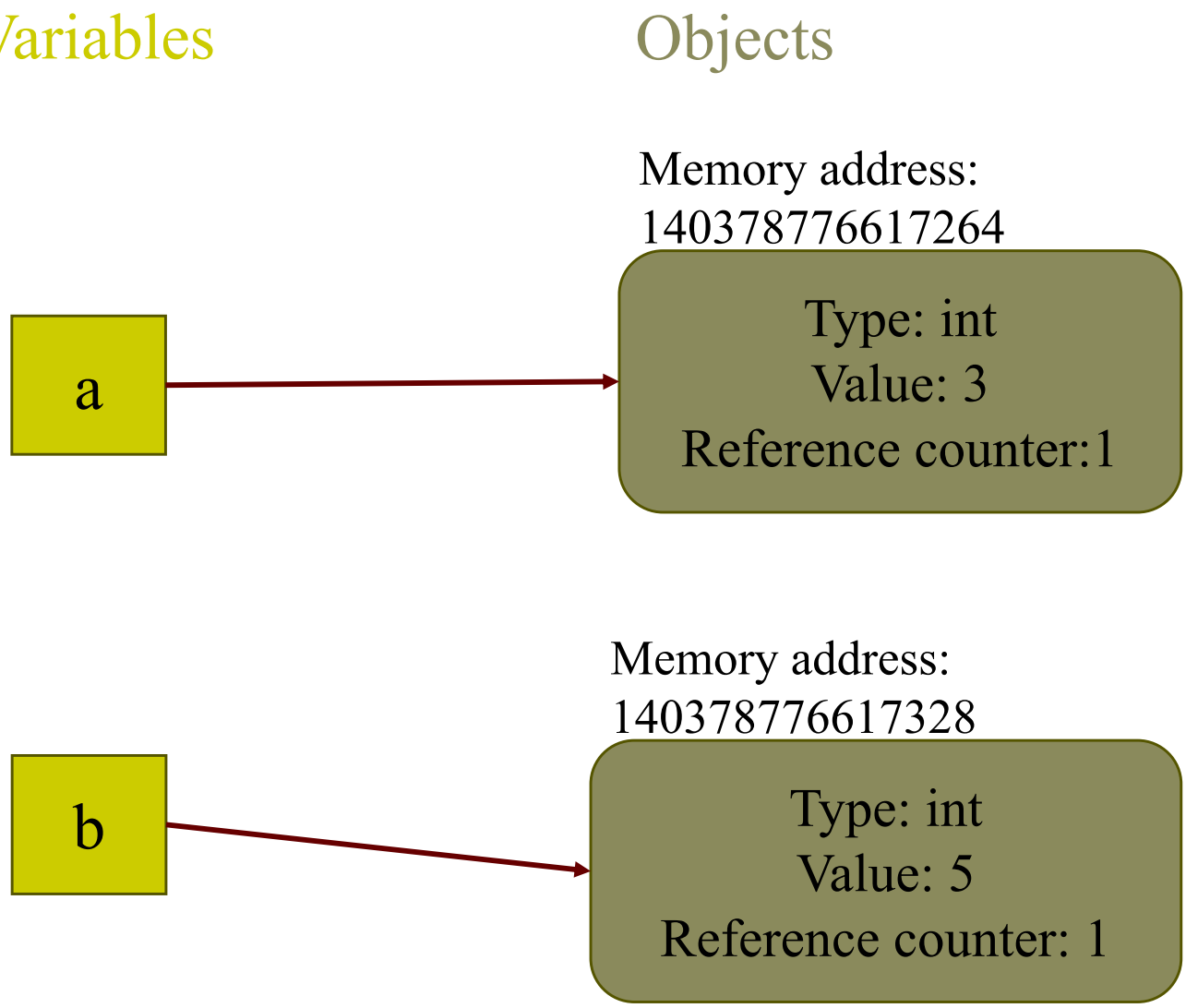
Data model

- Variables are just references to objects
  - Each variable contains the variable name and a pointer to an object
- Each object contains a reference counter that counts the number of variables that point to it
  - When the counter reaches zero, the object is marked for deallocation by the memory manager
- Variable assignment copies the reference, not the object
- The null object (None) can be assigned to any variable
- Making a copy of an object only makes sense for mutable objects:
  - The copy() function can be employed to copy lists

2. Fundamentals

Data model

```
>>> a = 5
>>> print(a)
5
>>> print(id(a))
140378776617328
>>> b = a
>>> a = 3
>>> print(a)
3
>>> print(id(a))
140378776617264
>>> print(b)
5
>>> print(id(b))
140378776617328
```



2. Fundamentals

Type model

- Variables do not have a static type (dynamic typing)
  - A variable has the type of the object it is currently pointing to
  - The type of a variable can change several times at runtime
  - Type declarations (type annotations) can optionally be done, but they just lead to compile or runtime warnings, i.e., they are not enforced by the interpreter
- However, we cannot operate variables of different types unless an explicit conversion is done (strong typing):

```
>>> print(5+"a")
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

2. Fundamentals

Booleans and integers

- The bool class only has two possible values: True and False
- Boolean operators: and, or, not
  - Short-circuit evaluation is employed for and, or
- The int class allows arbitrarily large values
- Integer arithmetic operators: +, -, *, //, %, **
- Assignment operators: =, +=, -=, *=, //=, %=, **=
- Comparison operators: ==, !=, >, <, >=, <=

2. Fundamentals

Floating-point numbers

- The float class represents double precision floating-point numbers
  - The arithmetic operator / and the assignment operator /= correspond to floating-point division
- Implicit conversions (promotions) are carried out for expressions mixing bool, int and float variables
  - False is converted to 0 and True is converted to 1
- Explicit conversions can be done with bool(), int() and float()

2. Fundamentals

Strings

- The str class represents strings (sequences) of Unicode code points (characters)
- String literals are enclosed in single or double quotes
  - You may use single quotes to enclose a string that contains double quotes, and vice versa
  - Multiline string literals can be specified with triple quotes
- The backslash \ is employed to escape special characters
  - Newline \n, tab \t
- Conversion to string: str()
- String length: len()
- Read a string from the console: input()
- String concatenation: +

2. Fundamentals

String indexing and slicing

- The individual characters in a string can be accessed by indexing in two ways:
  - Zero-based indices are available (index 0 corresponds to the first character)
  - Negative indices start from the end of the string backwards (index -1 corresponds to the last character)
- Substrings can be extracted by slicing:
  - Copy all the string, [:]
  - From start to the end of the string, [start:]
  - From the start of the string to end minus one, [:end]
  - From start to end minus one, [start:end]
  - From start to end minus one in steps of size step, [start:end:step]
  - Note that start and/or end can be positive, negative or zero
  - Note that step can be positive or negative, but not zero

2. Fundamentals

String operations

- Check for substrings: substring in string, substring not in string
- Split string by separator: string.split(',')
- Strip whitespaces: string.strip(), string.lstrip(), string.rstrip()
- Find index of first occurrence of substring:
  - Return -1 if not found: string.find(substring)
  - Raise exception if not found: string.index(substring)
- Count occurrences: string.count(substring)
- Repeat n times: string * n
- Replace substring: string.replace(old, new, num_replacements)
- String interpolation (f-strings):

```
>>> my_name = "John Smith"
>>> print(f"My name is: {my_name}\n")
My name is John Smith
```

3. FLOW CONTROL

3. Flow control

Code blocks and comments

- The code blocks are not marked by delimiters but by the indentation
- The number of whitespaces that mark a code block is usually 2 or 4
  - The criterion must be consistent across a source code file
- Comments are marked by #
  - They extend to the end of the line
- Document strings (docstrings) are defined by triple quotes
  - They should be placed right after a function or class definition
  - They should be descriptive
  - They can be accessed with help(function) or help(class)

3. Flow control

The if sentence

```
if condition:
    code_block

if condition:
    code_block1
else:
    code_block2

if condition1:
    code_block1
elif condition2:    # More elif clauses can follow
    code_block2
else:
    code_block3
```

3. Flow control

The match-case sentence

```
match expression:
    case pattern_1:
        code_block_1
    case pattern_2:
        code_block_2
…
    case pattern_n:
        code_block_n
    case _:     # If the expression does not match any of the previous patterns
        code_block_final
```

3. Flow control

The while sentence

- Basic loop structure:

```
while condition:
    code_block
```

- The continue sentence can be used to skip the remainder of the code block and evaluate the condition again
- The break sentence can be employed within the code block to terminate the loop immediately, before the condition is satisfied
- You can check whether the loop ended normally or by the execution of a break sentence:

```
while condition:
    code_block
else:
    code_block_else # Executes if the loop ended by execution of break
```

3. Flow control

The for sentence

- It iterates through an iterable object: string, list, dictionary, file, etc.
- Loop structure:

```
for element in iterable_object:
    code_block
```

- The break and continue sentences can be employed within the code block
  - The else clause can be added at the end to check whether the for loop ended by a break sentence
- The most frequent iterable object is a sequence of integers
  - The function range(start, end, step) is employed
  - The start parameter is optional, by default 0
  - The end parameter is mandatory, the sequence reaches end minus one
  - The step parameter is optional, by default 1



4. DATA STRUCTURES

4. Data structures

Lists

- Lists are mutable, i.e., they can be changed after they are created
- They store objects of any type with a defined order, duplications are allowed
- Lists are created by specifying the sequence of objects separated by commas and enclosed by square brackets:

```
>>> empty_list = []
>>> my_list = ["Python", 43, True, 43, -3.2836]
```

- The list() function can be used to convert other data types into lists:

```
>>> list("Python")
['P', 'y', 't', 'h', 'o', 'n']
>>> list(range(5))
[0, 1, 2, 3, 4]
```

4. Data structures

List indexing and slicing

- The syntax for list indexing and slicing is the same as strings:

```
>>> my_list[-2]
43
>>> my_list[1:5:2]
[43, 43]
```

- Indexing an element out of range results in an error
- Elements out of range are ignored when slicing, i.e., no error is thrown
- Length of the list: len(my_list)
- A copy of the list can be made with my_list[:]
- A reversed copy of the list can be made with my_list[::-1]

4. Data structures

List operations

- Copy the list: my_list.copy(), my_list.deepcopy()
- Reverse, keeping the original: reversed(my_list)
- Reverse, modifying the original: my_list.reverse()
- Append at the end: my_list.append(element)
- Insert at arbitrary position: my_list.insert(pos, element)
- Repeat n times: my_list * n
- Concatenate, keeping the original: +
- Concatenate, modifying the original: my_list.extend(list2), +=
- Modify/remove a sublist: my_list[start:end] = list2
  - list2 can have a different length from the substituted sublist
  - If list2==[], the sublist is deleted

4. Data structures

List operations

- Delete element by index: del my_list[index]
- Delete element by value (first occurrence): my_list.remove(element)
- Delete element by index, with extraction: element = my_list.pop(index)
- Clear the list, i.e., remove all elements: my_list.clear()
  - This is different from my_list = [], which creates a new empty list
- Check whether element exists: element in my_list
- Number of occurrences: my_list.count(element)
- Sort, keeping the original: sorted(my_list)
- Sort, modifying the original: my_list.sort()

4. Data structures

Iteration on lists

- Basic version:

```
for element in my_list:
    code_block
```

- Enumerate the elements starting with 0:

```
for index, element in enumerate(my_list):
    code_block
```

- Take one element of each list until the shortest list is exhausted:

```
for element1, element2 in zip(list1, list2):
    code_block
```

4. Data structures

Lists by comprehension

- Basic version:

my_list = [value for item in iterable]

>>> [character.upper() for character in "Python"]

['P', 'Y', 'T', 'H', 'O', ‘N’]

- Version with condition:

my_list = [value for item in iterable if condition]

>>> [c.upper() for c in "Python" if c>’m’]

['Y', 'T', 'O', 'N’]

- Nested version:

>>> [c1.upper()+c2.lower() for c1 in "Py" for c2 in "th"]

['Pt', 'Ph', 'Yt', 'Yh']

4. Data structures

Tuples

- Tuples are very similar to lists, only that tuples are immutable, i.e., they cannot be modified after they are created
  - Trying to modify a tuple results in a runtime error
- Tuples are created by specifying the sequence of objects separated by commas and enclosed by parentheses:

```
>>> empty_tuple = ()
>>> one_element_tuple = ("This element", )   # The comma is required
>>> other_tuple = (-5.2783, "Hello", True, 826)
```

- The parentheses can be omitted provided that there is at least one comma:

```
>>> one_element_tuple = "This element",    # The comma is required
>>> other_tuple = -5.2783, "Hello", True, 826
```

4. Data structures

Tuples

- Other iterable data types can be converted to tuples with the tuple() function:

```
>>> tuple(["One", "Two", "Three"])
("One", "Two", "Three")
```

- All operations defined for lists also work for tuples, except those that modify the original list
- Tuple unpacking into several variables:

```
>>> a, b, c = ("One", "Two", "Three")
```

- Variable swap by tuple unpacking:

```
>>> a, b = b, a
```

4. Data structures

Tuples

- Extended tuple unpacking allows grouping several elements of the unpacked tuple:

```
>>> my_tuple = ('G', 'A', 'R', 'Y', 'W')
>>> head, *body, tail = my_tuple
>>> head
'G'
>>> body
['A', 'R', 'Y']
>>> tail
'W'
```

- Unpacking can also be applied to other iterable data types

4. Data structures

Dictionaries

- Dictionaries (dicts) have the following characteristics:
  - They store values indexed by keys
  - They keep the order that the keys are inserted
  - They are mutable
  - The keys must be unique. Any immutable data type can be a key
  - The hash() builtin function is used to hash the keys
  - Access to the elements is very fast
- Dictionaries are created by surrounding a sequence of key:value assignments around curly brackets:

```
>>> empty_dict = {}
>>> other_dict = { "a": 572, "b": 826, "c": -4844, "d": 572}
```

4. Data structures

Dictionaries

- Elements are accessed by specifying the key between square brackets for reading, modifying or inserting an element:

```
>>> other_dict["d"]
572
```

- Accessing an inexistent key for reading results in a runtime error
  - Checking whether the key exists: key in my_dictionary, key not in my_dictionary
  - The get() function returns the value if the key exists, and None if the key does not exist
- Number of key-value pairs: len(my_dictionary)

4. Data structures

Dictionaries

- Obtain all keys: my_dictionary.keys()
- Obtain all values: my_dictionary.values()
- Obtain all key-value pairs as a list of tuples: my_dictionary.items()
- Iteration over a dict can be done with the previous three functions
- Two or more dicts can be merged
  - In case of key collision, the value of the last dict is kept
- Merge keeping the original dicts:

```
>>> {**dict1, **dict2}
>>> dict1 | dict2
```

- Merge modifying one of the dicts:

```
>>> dict1.update(dict2)
```

4. Data structures

Dictionaries

- Deleting by key:

```
>>> del my_dict[key]
```

- Deleting with extraction:

```
>>> element = my_dict.pop(key)
```

- Clear the entire dict (remove all key-value pairs):
  my_dict.clear()
  - This is different from my_dict = {}, which creates a new empty dict
- Dictionaries by comprehension:

```
>>> my_dict = {w: len(w) for w in words if w[0] not in "aeiou"}
```

4. Data structures

Sets

- A set contains several unique values without a specified order
- Sets are mutable
- The elements must be hashable
- A set can be created by separating the values by commas and enclosing with curly braces:

```
>>> numbers = { 67, -34, 82, -111}
```

- The empty set must be created with set() without arguments
- Other iterable data types can be converted to sets by the set() function
  - Duplicate values are removed prior to set creation

4. Data structures

Sets

- Element addition: my_set.add(element)
- Element removal: my_set.remove(element)
- Number of elements (cardinal): len(my_set)
- Check whether an element exists: element in my_set, element not in my_set
- Freeze a set (make it immutable): frozenset(my_set)
- Iterate over a set: for element in my_set
- Set operations: intersection (&), union (|), difference (-), symmetric difference (^), inclusion (<, <=, >, >=)
- Sets by comprehension:

```
>>> my_set = {n for n in range(0, 20) if n % 3 == 0}
```

5. MODULARITY

5. Modularity

Functions

- Function definition with its parameter list:

```
def function_name(parameter1, parameter2):
    function_body
```

- The function body must contain at least one sentence
  - The pass sentence is a no-operation
- The return sentence can be used to return a value, otherwise None is returned
  - If return is used without a return value, again None is returned
- Multiple values can be returned as a tuple
- Function call with its argument list:

```
function_name(argument1, argument2)
```

5. Modularity

Functions

- Positional arguments are those that are assigned to their corresponding parameters by order
- Nominal arguments are copied by assigning each parameter by its name:

function_name(parameter2 = argument1, parameter1 = argument2)

- Positional and nominal arguments can be mixed but the positional arguments must go first
- Default values for the parameters can be specified in the function definition

def function_name(parameter1, parameter2 = 0.0):

5. Modularity
Functions

- Argument passing is done by "Call by Object Reference" also known as "Call by assignment"
- Each argument is assigned to its corresponding parameter like in a standard variable assignment, i.e., a reference to the object is copied
- For immutable objects this means that modifications in the parameters inside the function are not seen outside
- For mutable objects this means that modifications in the objects referenced by the parameters are seen outside
- If you assign a new object to a parameter inside the function, then the change is not seen outside

5. Modularity
Functions

- Documentation should be added to the definition of each function by inserting a docstring between triple quotes at the beginning of the function body:

```
def my_function(param1, param2):
    """ This is the docstring of the function

    …
    """
    rest_of_function_body
```

- The help() function can be used to display the docstring of a function:

```
>>> help(my_function)
```

5. Modularity
Functions

- Type hints can be added to indicate the expected types for the parameters and the return value

```
def my_function(param1: str, param2: int) -> dict[str, float]:
```

- Variables can also be given hints, with or without assignment:

```
>>> age: int = 1
>>> year: int
```

- The Python interpreter does not enforce function and variable type annotations
  - They can be used by third party tools such as type checkers, IDEs, linters, etc.

5. Modularity
Object oriented programming

- Instance methods have a first parameter self that references the current instance
- Class methods have a first parameter cls that references the current class
  - The @classmethod decorator must be specified before the method definition
- The constructor is the __init__ instance method
- Instance attributes can be accessed with self.attribute_name from inside instance methods, or object_name.attribute_name from outside instance methods
  - They are usually defined inside the constructor
- Class attributes can be accessed with cls.attribute_name from inside class methods, or class_name.attribute_name from outside class methods
  - They are usually defined outside the constructor

5. Modularity
Object oriented programming

```
class Droid:
    droid_counter = 0

    @classmethod
    def num_droids(cls) -> int:
        return cls.droid_counter

    def __init__(self):
        self.power_on = False
        Droid.droid_counter += 1

    def switch_on(self):
        self.power_on = True

    def switch_off(self):
        self.power_on = False
```

5. Modularity

Modules

- A module is a .py source file
- Modules can be grouped in folders called packages
- Folders can be grouped into libraries
- Sometimes the “package” and “library” terms are used interchangeably
- Modules can be imported entirely:

```
>>> import stats
>>> stats.std(6, 3, 9, 5)
2.5
```

- You can also import specific functions from a module:

```
>>> from stats import mean
>>> mean(6, 3, 9, 5)
5.75
```

5. Modularity

Modules

- Aliases can be used for modules or functions:

```
>>> from stats import mean as avg
>>> avg(6, 3, 9, 5)
5.75
>>> import stats as mystatistics
>>> mystatistics.std(6, 3, 9, 5)
2.5
```

- The main module in a project is usually named main.py
- Entry point of the main program:

```
if __name__ == '__main__':
    # This is the entry point of the main program
```

- Each module can be either imported or executed as a main program

6. CONCLUSION

6. Conclusion

- Some features of the language must be managed with care to avoid hard-to-spot bugs:
  - Dynamic typing, i.e., a variable data type may change at runtime
  - Data structures can hold elements of mixed data types
  - Name collisions when importing libraries
- Tips to enhance the performance:
  - Use list comprehension rather than loops
  - Employ the builtin functions
  - Use “in” whenever possible
  - Use sets rather than loops
  - Use tuple unpacking